

"Smart Retail Analytics Hub" – Hybrid Data System for Real-Time and Historical Insights

ISM 4545 Big Data for AI and Analytics

Dr. Samuel S. Conn

Anastasiia Putilova, Anh Thai, Bao Han Phan, Elizaveta Savanina

November 10th, 2025

2. Executive Summary

We built a hybrid transactional/analytical data platform for a fictional retail chain that unifies day-to-day store operations with historical analytics and JSON-based views suitable for APIs and dashboards. The system consists of: (1) a normalized OLTP schema (Stores, Customers, Products, Inventory, Orders, Order_Items); (2) an OLAP star schema with Fact_Sales and dimensions (Dim_Product, Dim_Customer, Dim_Store, Dim_Time); and (3) a JSON document store plus a simulated JSON Relational Duality View that returns nested customer-order documents.

Using Oracle (Live SQL / 23c-compat) SQL features, we created and populated all tables, enforced keys and constraints, computed `orders.total_amount` from line items, and loaded the star schema from OLTP. We then executed analytical queries (time hierarchy, product/category, store/location, top customers, and CUBE aggregation). For JSON, we stored semi-structured customer profiles and exposed an API-style consolidated customer-orders view via `JSON_OBJECT` and `JSON_ARRAYAGG`, simulating Oracle 23ai duality functionality.

Key outcomes: working OLTP + OLAP + JSON layers, example queries across all layers; and clear paths to BI/AI use cases such as frequently-bought-together analysis and current-day sales dashboards.

3. Introduction

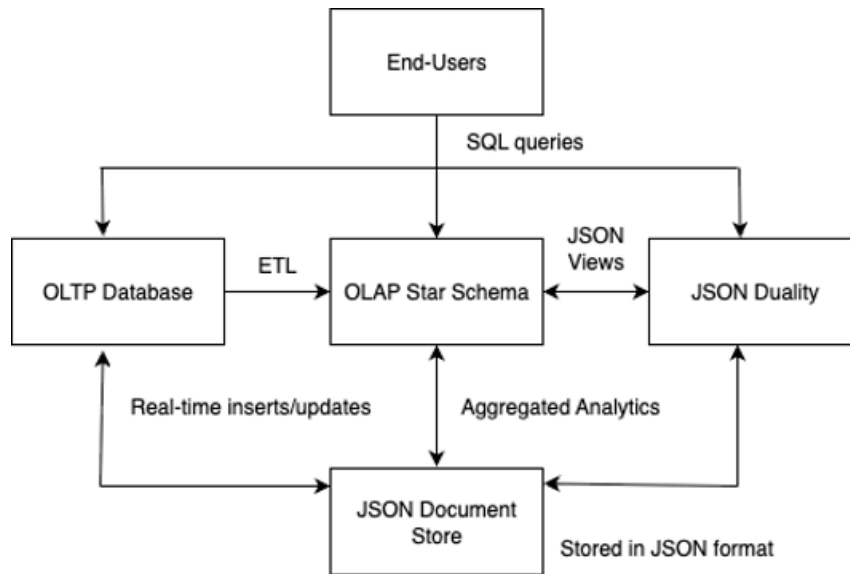
Context: Retail organizations must run reliable transactions while analyzing longer-term trends (categories, locations, and customers).

Goal and scope: Our goal was to design and implement an end-to-end data backbone that supports transactions, dimensional analytics, and JSON views for application consumption.

Summary of approach taken: We implemented a normalized OLTP model, transformed it to a star schema, and added JSON storage and a simulated duality view demonstrating a hybrid platform ready for BI and AI.

4. System Architecture and Design

High-level system architecture diagram



The high-level system architecture of “Smart Retail Analytics Hub” is designed to handle both day-to-day operations and analytics. It combines transactional sales data from stores and customers in real time and ensures that orders, inventory, and product information are up to date. The system transforms this data into an analytical layer that supports sales reporting, analysis of trends, and insights into high-performing products. Moreover, the platform provides AI-ready data, which allows for personalization and predictive analytics. The design enables the company to operate efficiently while making data-driven decisions for marketing, inventory management, and customer engagement.

Description of components:

- **Relational schema (OLTP)**

The OLTP layers store operational data in a normalized relational database, which allows for placing orders, managing inventory, and maintaining customer information.

Table	Description
Customers	Stores customer details: ID, name and contact information
Products	Stores product catalog: product ID, name of the product, SKU, category and price.
Stores	Contains information about stores: name and location
Inventory	Tracks quantities of products and which of them are stocked in which stores.
Orders	Records transactions, order status and totals

Order_items	Records products included in each order
-------------	---

The OLTP schema is also normalized to Third Normal Form (3NF) which removes redundancy and supports data integrity. Primary keys ensure that each record is unique. Foreign key relationships maintain that each ordered item references legit order and product. The schema also acts as a foundation for analytical transformations into the OLAP layer.

- ***OLAP star schema and cubes***

The OLAP layer is built to help the business analyze and understand its data. It takes normalized transactional data from the OLTP system and organizes it into a star schema, which is a simpler, denormalized structure. The OLAP layer turns raw operational data into meaningful insights for decision-making and strategic planning.

- ***JSON document store***

The JSON document store extends the system to handle semi-structured and evolving data that does not fit neatly into fixed tables. Customer profiles and order details are stored as JSON documents keyed by the same IDs as the OLTP tables, so they can still be joined back to relational data. This flexible layer allows new attributes (such as preferences, loyalty behavior, or extra metadata) to be added without changing the schema, and it provides richer, nested data that can be used for advanced analytics, personalization, and future AI models.

- ***JSON Relational Duality Views***

JSON Relational Duality Views provide a unified way to work with both relational tables and JSON documents. They expose customer and order-centric JSON objects that are still backed by normalized tables and the JSON document store, so applications can query and update data as if it were a single JSON structure. This makes it easier for front-end apps and APIs to consume rich, nested data while the database still benefits from the performance, integrity, and flexibility of the underlying OLTP, OLAP, and JSON layers.

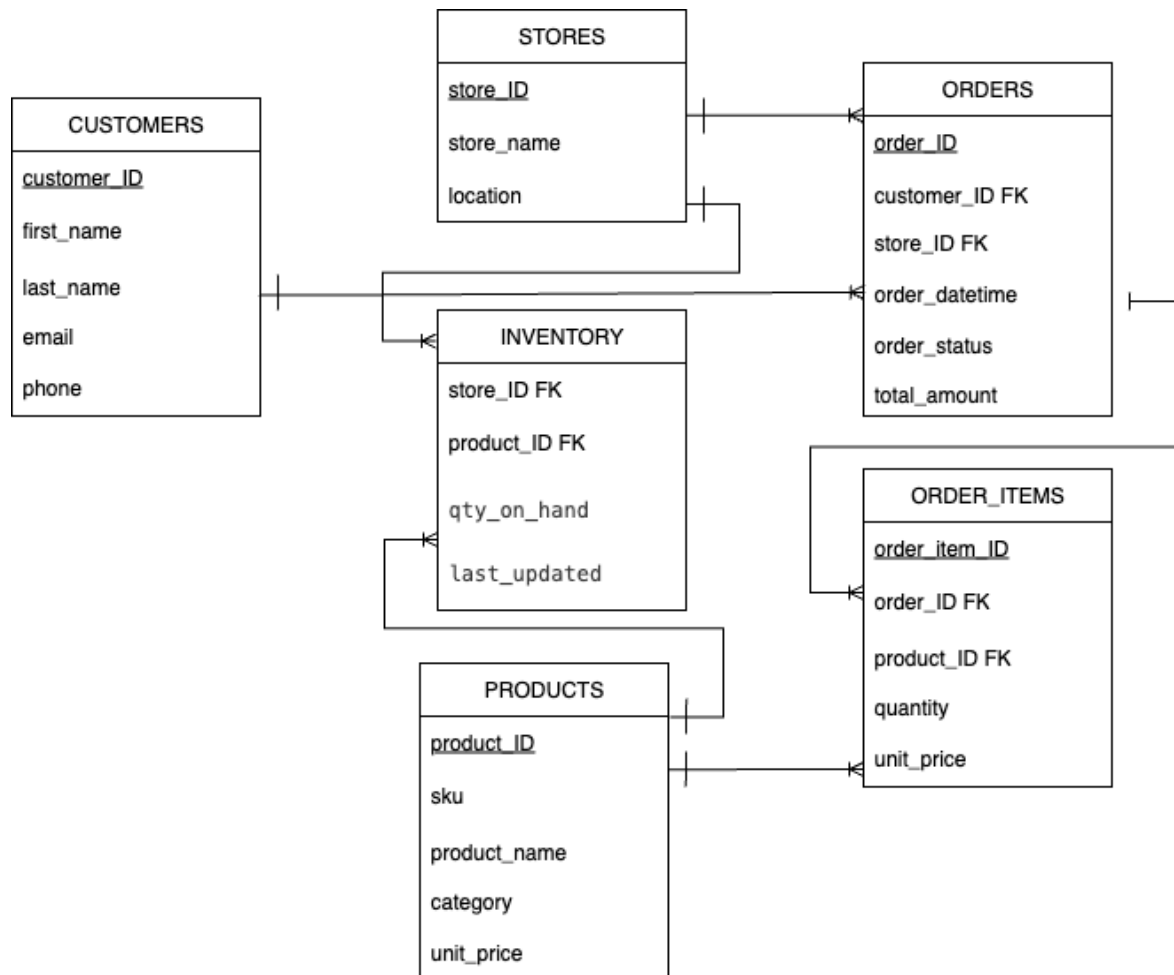
Justification of Design Decisions

The overall architecture intentionally separates transaction processing, analytics, and semi-structured data while still keeping them tightly integrated. The OLTP layer uses a normalized relational schema so day-to-day operations (creating customers, recording orders, updating inventory) are ACID-compliant, consistent, and fast. The OLAP layer then reshapes this data into a star/snowflake style structure optimized for aggregation and time-series analysis, so dashboards and analytics queries do not put heavy load on the transactional system. On top of that, the JSON document store gives us flexibility for attributes that change frequently or are not known in advance (such as customer preferences, loyalty behavior, or detailed line-item metadata), avoiding constant schema changes while still keeping documents keyed to the same IDs as the relational tables. Finally, JSON Relational Duality Views provide a unified customer-centric JSON view backed by those relational and JSON structures, allowing external applications and BI tools to work with rich JSON documents without sacrificing the integrity, performance, and query power of SQL underneath. Together, these design choices balance consistency and performance with flexibility and future scalability, especially for advanced analytics and AI use cases.

5. Implementation

5.1: Relational Database

i.ER Diagram



The ER diagram illustrates the head structure of the transactional database for the Smart Retail Analytics Hub. It shows how data about customers, stores, products, inventory, and orders is organized and connected. Each entity in the diagram contains essential details about customers, stores, inventories, products, orders and ordered items. The schema is designed according to 3NF, thus all data is structured to avoid redundancy and inconsistencies.

Entity	Description	Primary Key	Relationship
Customers	Stores customer information (first name, last name, email, phone)	customer_id	One-to-Many (1:N) with Orders (one customer can place many orders)
Stores	Shows store locations	store_id	One-to-Many (1:N) with Inventory (each

			store has multiple products) One-to-Many (1:N) with Orders (orders are placed at one store)
Products	Contains product details (SKU, name, category, unit price)	product_id	One-to-Many (1:N) with Inventory (each product can exist in multiple stores) One-to-Many (1:N) with Order_Items (products sold in multiple orders)
Inventory	Tracks the quantity of each product at each store	store_id, product_id (composite key)	Many-to-One (N:1) with Stores Many-to-One (N:1) with Products
Orders	Represents customer purchases	order_id	Many-to-One (N:1) with Customers Many-to-One (N:1) with Stores One-to-Many (1:N) with Order_Items
Order_Items	Represents individual products within an order	order_item_id	Many-to-One (N:1) with Orders Many-to-One (N:1) with Products

- One customer can place many orders but each order belongs to only one customer.
- One store holds many products in its inventory, and the inventory table records these quantities. Each inventory record references one specific store.
- One store can have many orders but each order refers to only one store
- Inventory can contain products that are available in many stores but each inventory record refers to one specific product.
- One product can appear in many orders but each order item references one product.
- The inventory table acts as a bridge between stores and inventory tables representing the Many-to-Many relationship. One store can stock many products and vice versa. Also, the inventory table utilizes composite primary key - store_ID and product_ID to represent unique store_product combinations.
- One order can have multiple products but each order item belongs to only one order.
- Each ordered item refers to one product, showing the exact product sold and revenue gained.

ii. Normalized schema description and SQL table (metadata) definitions

The relational schema was formatted according to Third Normal Form (3NF) ensures that all data fields rely only on the primary keys and no partial dependencies exist. Each entity represents a real-world component essential to retail business: customers, stores, inventory orders and ordered items. Relationships between entities are enforced through the implementation of foreign key constraints. The database also promotes referential integrity through key-based relationships eliminating inconsistencies, duplicates or orphaned items.

Customers table stores information about individual customers, such as name, contact details and unique identifiers:

Column Name	Data Type	Description	Constraints
customer_id	NUMBER	Unique identifier for a customer	Contact email
first_name	VARCHAR2(50)	Customer's first name	Not Null
last_name	VARCHAR2(50)	Customer's last name	Not Null
email	VARCHAR2(100)	Contact email	Unique

```
CREATE TABLE customers (  
    customer_id NUMBER GENERATED BY DEFAULT AS IDENTITY  
PRIMARY KEY,  
    first_name  VARCHAR2(100) NOT NULL,  
    last_name   VARCHAR2(100) NOT NULL,  
    email       VARCHAR2(255) UNIQUE,  
    phone       VARCHAR2(50) ,  
    created_at  TIMESTAMP DEFAULT SYSTIMESTAMP  
);
```

Stores table contains information about each store including names and locations:

Column Name	Data Type	Description	Constraints
store_id	NUMBER	Unique store identifier	Primary Key
store_name	VARCHAR2(100)	Name of store branch	Not Null
location	VARCHAR2(150)	Store geographic location	Not Null

```

CREATE TABLE stores (
    store_id    NUMBER GENERATED BY DEFAULT AS IDENTITY
PRIMARY KEY,
    store_name VARCHAR2(100) NOT NULL,
    location   VARCHAR2(200) ,
    created_at  TIMESTAMP DEFAULT SYSTIMESTAMP
);

```

Products table represents all items that are available for sale:

Column Name	Data Type	Description	Constraints
product_id	NUMBER	Unique product identifier	Primary Key
sku	VARCHAR2(50)	Stock Keeping Unit	Unique, Not Null
product_name	VARCHAR2(100)	Name of product	Not Null
category	VARCHAR2(50)	Product category	Not Null
unit_price	NUMBER(10,2)	Price per unit	Not Null

```

CREATE TABLE products (
    product_id  NUMBER GENERATED BY DEFAULT AS IDENTITY
PRIMARY KEY,
    sku         VARCHAR2(50) UNIQUE NOT NULL,
    product_name VARCHAR2(200) NOT NULL,
    category    VARCHAR2(50) ,
    unit_price  NUMBER(10,2) NOT NULL
);

```

Inventory table links stores and products tables, highlighting stock levels at each store unit :

Column Name	Data Type	Description	Constraints
store_id	NUMBER	Store where product is stocked	Part of Primary Key, Foreign Key to stores
product_id	NUMBER	Product being stocked	Part of Primary Key, Foreign Key to products
qty_on_hand	NUMBER(10)	Available stock count	Default 0
last_updated	TIMESTAMP	Timestamp of last update	Default current time

```

CREATE TABLE inventory (
    store_id  NUMBER NOT NULL,
    product_id  NUMBER NOT NULL,
    qty_on_hand  NUMBER(10) DEFAULT 0,
    last_updated  TIMESTAMP DEFAULT SYSTIMESTAMP,
    CONSTRAINT pk_inventory PRIMARY KEY (store_id,
product_id),
    CONSTRAINT fk_inventory_store FOREIGN KEY (store_id)
REFERENCES stores(store_id),
    CONSTRAINT fk_inventory_product FOREIGN KEY (product_id)
REFERENCES products(product_id)
);

```

Orders table shows purchase transactions, time and in which store purchase was made and total value::

Column Name	Data Type	Description	Constraints
order_id	NUMBER	Unique order identifier	Primary Key
customer_id	NUMBER	Customer placing the order	Foreign Key to customers
store_id	NUMBER	Store fulfilling the order	Foreign Key to stores
order_datetime	TIMESTAMP	Time order was placed	Default current time
order_status	VARCHAR2(30)	Status (PLACED, SHIPPED, etc.)	Default 'PLACED'
total_amount	NUMBER(12,2)	Monetary value of order	Default 0

```

CREATE TABLE orders (
    order_id  NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY
KEY,
    customer_id  NUMBER REFERENCES customers(customer_id),
    store_id  NUMBER REFERENCES stores(store_id),
    order_datetime  TIMESTAMP DEFAULT SYSTIMESTAMP,
    order_status  VARCHAR2(30) DEFAULT 'PLACED',
    total_amount  NUMBER(12,2) DEFAULT 0
);

```

Order_items table records product quantity and pricing of each order:

Column Name	Data Type	Description	Constraints
order_item_id	NUMBER	Unique identifier for order line	Primary Key
order_id	NUMBER	Associated order	Foreign Key to orders
product_id	NUMBER	Product sold	Foreign Key to products
quantity	NUMBER(10)	Number of units purchased	Default 1
unit_price	NUMBER(10, 2)	Price per unit at sale time	Not Null

```
CREATE TABLE order_items (  
    order_item_id NUMBER GENERATED BY DEFAULT AS IDENTITY  
    PRIMARY KEY,  
    order_id      NUMBER REFERENCES orders(order_id) ON  
DELETE CASCADE,  
    product_id    NUMBER REFERENCES products(product_id) ,  
    quantity      NUMBER(10) DEFAULT 1,  
    unit_price    NUMBER(10,2) NOT NULL  
);
```

iii. Sample transactional queries

1. Inserting new customer record in the system

```
INSERT INTO customers (first_name, last_name, email, phone)  
VALUES ('Sarah', 'Lopez', 'sarah.lopez@mail.com',  
' +1-555-0111');
```

This command shows how new customer information can be inserted in the OLTP system.

2. Creating a new order

```
INSERT INTO orders (customer_id, store_id, order_status)  
VALUES (11, 2, 'PLACED');
```

This command shows a transaction in the system when a customer places an order.

3.Viewing order details with Joins

```
SELECT o.order_id, c.first_name, s.store_name, oi.product_id,  
oi.quantity, oi.unit_price  
FROM orders o  
JOIN customers c ON o.customer_id = c.customer_id  
JOIN stores s ON o.store_id = s.store_id  
JOIN order_items oi ON o.order_id = oi.order_id  
WHERE o.order_id = 5;
```

This command pulls all the details about the order by linking information from other tables.

4.Updating order status

```
UPDATE orders  
SET order_status = 'SHIPPED'  
WHERE order_id = 5;
```

This command is used when updating the status of the order when it's being proceeded to next stages.

5.Deleting old record

```
DELETE FROM inventory  
WHERE store_id = 10 AND product_id = 6;
```

This command is used to remove the product that is no longer carried in the store. This promotes referential integrity and eliminates the probability of incorrect stocking or ordering.

6.Retrieving the total price

```
UPDATE orders o  
SET total_amount = (  
    SELECT SUM(quantity * unit_price)  
    FROM order_items  
    WHERE order_id = o.order_id  
)  
WHERE order_id = 5;
```

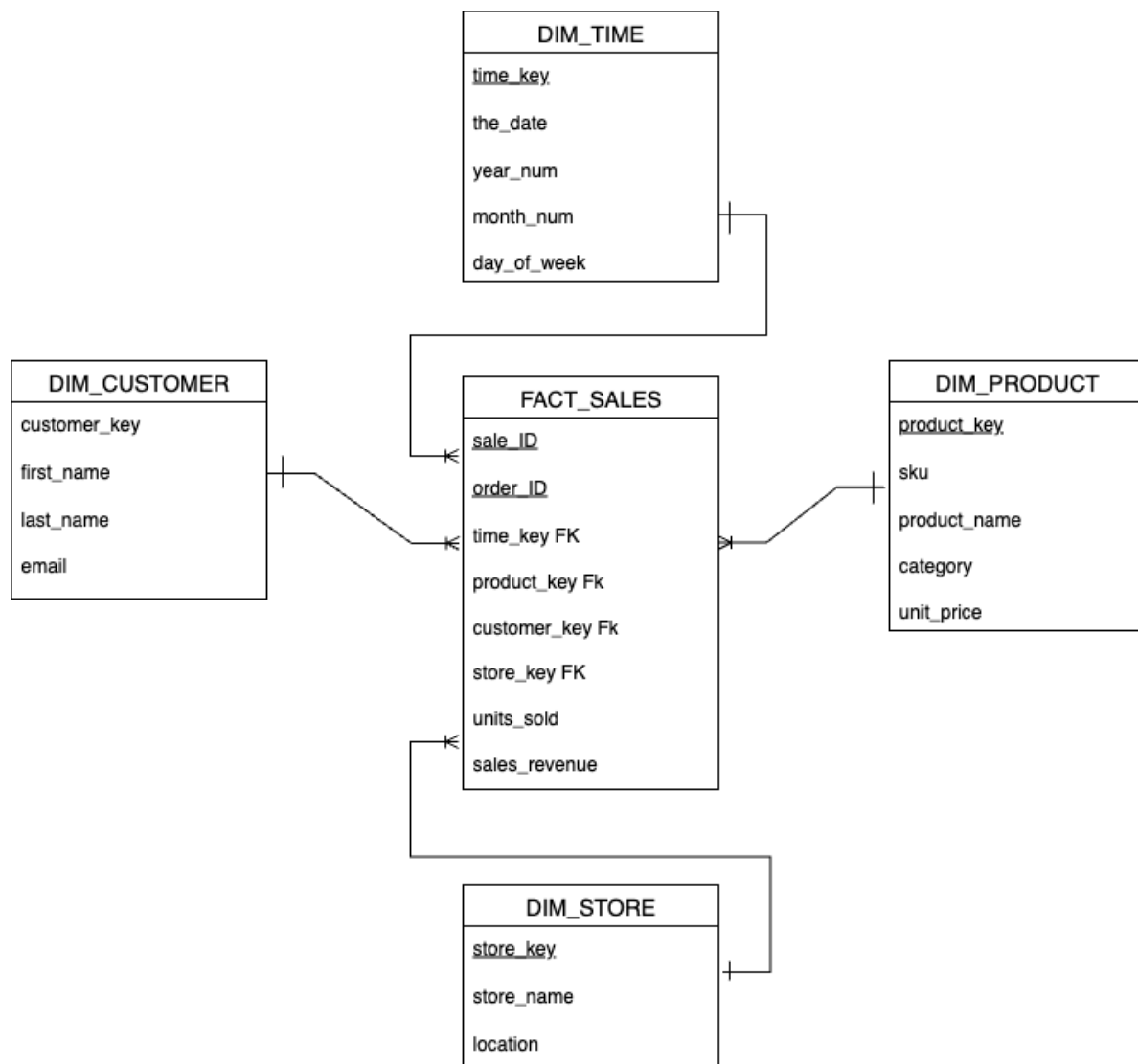
This command ensures that the order's total price is correct and matches the items purchased.

These queries together illustrate how the relational database contributes to handling real-time high-frequency transactions efficiently. The normalized OLTP structure ensures data consistency and eliminates redundancy during inserts, updates, and retrievals.

5.2: OLAP Star Schema

i. Star schema diagram

The star schema is a denormalized data model optimized for analytical queries and reporting.



ii. Fact and dimension tables with descriptions

Our star schema is centered around a fact_sales table connected with dim_product, dim_customer, dim_store, and dim_time dimensional tables. Dimension tables store descriptive attributes (such as product categories, store locations, and dates) that allow for filtering, grouping, and analyzing the numerical data stored in the fact table. This enables

insights like revenue by store, sales by month, or customer buying patterns. This structure allows fast performance for aggregation, trend analysis, and data exploration.

Table	Description
fact_sales (Fact)	Sales transactions with measures for units_sold and sales_revenue.
dim_product	Product details like name, category, stock, and unit price.
dim_customer	Customer details including first name, last name, and email
dim_store	Store information such as location and name
dim_time	Time attributes used for the time-series analysis: year_num, month_num, day_of_week

iii. Data Cube Measures and Hierarchies

The three main hierarchies were established in the Smart Retail Analytics Hub OLAP environment to support multi-dimensional analysis - Time, Product, and Store. These hierarchies were chosen to reveal hidden sales trends and performance patterns across time, product, and geographical areas.

Time hierarchy (Year to Month to Day)

This dimension is used to look at revenue trends over time.

Query:

```
SELECT dt.year_num, dt.month_num,
       SUM(fs.sales_revenue) AS total_revenue
FROM fact_sales fs
JOIN dim_time dt ON fs.time_key = dt.time_key
GROUP BY dt.year_num, dt.month_num
ORDER BY dt.year_num, dt.month_num;
```

Resulting output shows total revenue grouped by year and month. In our specific case the result is - for November 2025, total revenue = 143.05.

If we were to populate our tables over a longer period of time, this query would have allowed us to see patterns like customer demand and seasonality trends. Such insights are very valuable since they help make managerial decisions related to marketing promotions and

inventory planning for example. Having such hierarchies in the cube, helps managers to predict trends and adjust operations accordingly.

Product Hierarchy. (Category to Product)

This dimension is used to look at connection between product and category, which helps understand the contribution of product lines to the total sales.

Query:

```
SELECT dp.category, dp.product_name,
       SUM(fs.units_sold) AS total_units,
       SUM(fs.sales revenue) AS total revenue
FROM fact_sales fs
JOIN dim_product dp ON fs.product_key = dp.product_key
GROUP BY dp.category, dp.product_name
ORDER BY dp.category, dp.product_name;
```

Result:

	CATEGORY	PRODUCT_NAME	TOTAL_UNITS	TOTAL_REVENUE
1	beverages	Apple Juice 1L	5	19.95
2	beverages	Coffee Beans 500g	2	25.98
3	beverages	Green Tea 250g	4	33.96
4	beverages	Orange Juice 1L	5	17.95
5	dairy	Cheddar Cheese 200	3	14.97
6	dairy	Milk 1L	4	9.96
7	dairy	Yogurt 500g	2	6.2
8	snacks	Chocolate Bar	4	5
9	snacks	Granola Bar	3	4.5
10	snacks	Potato Chips	2	4.58

From the output we got it is evident that beverages produce most of the revenue in our stores, especially green tea and coffee beans. Snacks on the other hand produce the smallest amount of revenue. Information like this is useful to managers because they now can give preference to beverage marketing or adjust product positioning in store to draw attention to high-margin products. Such information can also help decide on which categories to expand on or which products to substitute in the future by looking at the contribution ratios.

Store Hierarchy. (Location to Store)

This dimension is used to look at stores' performance across different geographic areas.

Query:

```
SELECT ds.location, ds.store name,  
       SUM (fs.sales_revenue) AS total_revenue  
FROM fact_sales fs  
JOIN dim_store ds ON fs.store_key = ds.store_key  
GROUP BY ds. location, ds.store_name  
ORDER BY ds.location, ds.store_name;
```

Result:

	LOCATION	STORE_NAME	TOTAL_REVENUE
1	111 Lake St	QuickBuy	7.98
2	123 Game St	The Store	9.48
3	222 Elm St	BudgetMart	12.02
4	321 Market Rd	FreshMart	9.47
5	333 Oak Blvd	ShopEZ	12.27
6	444 Pine Rd	FamilyStore	20.08
7	456 Highway	Cool Store	17.97
8	654 Main Blvd	DailyShop	20.08
9	789 Event Ave	Super Store	8.74
10	987 Grand Ave	MegaStore	24.96

Based on the results we can see that certain areas are clearly underperformed compared to other ones. As an example interpretation we can assume that the information pointed out that high revenues were mainly coming from urban places with a lot of traffic, for instance, MegaStore, and at the same time, smaller stores such as QuickBuy were not very successful. In reality, of course, we would have had actual locations and revenues which could have pointed to specific patterns by location. Such information helps managers decide on the location of the store, marketing efforts, stocking expansion, customer attraction strategies etc.

iv. Example analytical queries and insights

We have created an example of analytical query to showcase how the cube could be used to retrieve actionable insights for business decisions.

The first query is designed to return top 5 customers by their total spending.

Query:

```
SELECT dc.first_name || ' ' || dc.last_name AS customer_name,  
       SUM(fs.sales_revenue) AS total_spent
```

```

FROM fact_sales fs

JOIN dim_customer dc ON fs.customer_key = dc.customer_key

GROUP BY dc.first_name, dc.last_name

ORDER BY total_spent DESC

FETCH FIRST 5 ROWS ONLY;

```

Result:

	CUSTOMER_NAME	TOTAL_SPENT
1	Frank Lee	24.96
2	Emily Clark	20.08
3	Jake Stone	20.08
4	Bob Khan	17.97
5	Isla Patel	12.27

This analytical query helps us to see that Frank Lee and Emily Clark are the top spenders among customers. Figuring out information like this helps in customer relationship management. This opens opportunities like targeted loyalty programs or early access offers for high-value customers for high spenders and retention campaigns or cross-selling efforts for small spenders. Information like this over time can also accumulate into useful insights on impact of marketing effort and strategies.

Another query that we have run was the CUBE query, which allows for simultaneous aggregation across all major dimensions in order to highlight possible broad business patterns.

Query:

```

SELECT NVL(dp.category, 'ALL CATEGORIES') AS category,
       NVL(ds.location, 'ALL LOCATIONS') AS location,
       NVL(TO_CHAR(dt.year_num), 'ALL YEARS') AS year,
       SUM(fs.sales_revenue) AS total_sales

FROM fact_sales fs

JOIN dim_product dp ON fs.product_key = dp.product_key

JOIN dim_store ds ON fs.store_key = ds.store_key

JOIN dim_time dt ON fs.time_key = dt.time_key

GROUP BY CUBE(dp.category, ds.location, dt.year_num)

ORDER BY dp.category, ds.location, dt.year_num;

```

This query returns a lengthy table which aggregates across 3 dimensions (category, store location, and year). Instead of null you would see “all categories”, “all locations”, and “all years” for easier interpretation. Since the whole table is too large, here is a snippet of the selected rows, so that the general structure of the table is seen.

Category	Location	Year	Total Sales
Beverages	123 Game St	2025	7.98
Beverages	ALL LOCATIONS	2025	97.84
Beverages	ALL LOCATIONS	ALL YEARS	97.84
Dairy	789 Event Ave	2025	9.98
Dairy	ALL LOCATIONS	2025	31.13
Dairy	ALL LOCATIONS	ALL YEARS	31.13
Snacks	321 Market Rd	2025	5.00
Snacks	ALL LOCATIONS	2025	14.08
Snacks	ALL LOCATIONS	ALL YEARS	14.08
ALL CATEGORIES	111 Lake St	2025	7.89
ALL CATEGORIES	ALL LOCATIONS	2025	143.05
ALL CATEGORIES	ALL LOCATIONS	ALL YEARS	143.05

Looking at the table we can see that beverages have performed well across all stores, while stacks didn’t do as well. If we would have had real data with actual stores and location and if we were to populate it across several years, we would have been able to get trends and insights into things like preferred categories by location(possible demographic preference), or trends of specific categories over the years etc.. The cube-dimensional slicing could also be performed to look at such trends more comfortably, like viewing beverage sales in a single year, across all stores, or filtering by product type and location at the same time. This makes cube queries incredibly useful in business decision making.

5.3 JSON Document Store

i. Structure of JSON documents

In this phase, we designed two JSON-based tables to store semi-structured customer and order information within our Smart Retail Analytics Hub system.

```
CREATE TABLE customer_profiles (  
  customer_id NUMBER PRIMARY KEY,  
  profile_data JSON  
);  
  
CREATE TABLE order_history (  
  order_id NUMBER PRIMARY KEY,  
  order_data JSON  
);
```

Each record in these tables stores one JSON document, which allows us to manage flexible attributes such as customer loyalty points, preferences, and favorite stores without altering the relational schema.

Example JSON document & queries

```
{  
  "customer_id": 1,  
  "name": "Alice Mason",  
  "loyalty_points": 1200,  
  "preferences": {  
    "categories": ["snacks", "beverages"],  
    "favorite_store": "The Store"  
  }  
}
```

ii. Use cases for unstructured or semi-structured big data

We used the JSON document store to handle customer data that is not always consistent or easily modeled in fixed relational columns. This approach enables us to:

- Store detailed and dynamic customer information such as preferences, loyalty levels, and purchase habits.
- Support marketing personalization and product recommendation features.
- Combine structured and unstructured data for deeper customer behavior analysis.

- Prepare the system for AI-driven analytics and predictive modeling in future enhancements.

iii. Example JSON documents and queries

We inserted and queried JSON data directly in Oracle Live SQL to demonstrate this functionality.

```
INSERT INTO customer_profiles VALUES (
  1,
  JSON_OBJECT(
    'customer_id' VALUE 1,
    'name' VALUE 'Alice Mason',
    'email' VALUE 'alice@mail.com',
    'preferences' VALUE JSON_OBJECT('favorite_category' VALUE 'beverages')
  )
);

SELECT customer_id, profile_data FROM customer_profiles;
```

This query successfully returned the stored JSON object, confirming that Oracle Live SQL can store and retrieve semi-structured data as part of the hybrid retail analytics environment.

5.4 JSON Relational Duality Views

i. Description and purpose of each view

Oracle 23ai introduces *JSON Relational Duality Views*, which combine relational and JSON data models into a unified representation.

Since Oracle Live SQL does not currently support this feature, we created a simulated duality view using SQL JSON functions such as `JSON_OBJECT` and `JSON_ARRAYAGG`.

The purpose of this view is to display each customer along with all of their associated orders in a single, nested JSON document, allowing us to query relational data as if it were stored in a document format.

ii. SQL definition/syntax of views

We created the following simulated view to represent customer-order relationships in JSON format:

```

CREATE OR REPLACE VIEW customer_orders_json_v AS
SELECT
  c.customer_id,
  c.first_name,
  c.last_name,
  c.email,
  JSON_ARRAYAGG(
    JSON_OBJECT(
      'order_id' VALUE o.order_id,
      'order_date' VALUE TO_CHAR(o.order_datetime, 'YYYY-MM-DD'),
      'total_amount' VALUE o.total_amount
    )
  ) AS orders_json
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id
GROUP BY c.customer_id, c.first_name, c.last_name, c.email;

```

This view aggregates each customer's orders as a JSON array, producing a document-like structure that closely simulates the duality view feature of Oracle 23ai.

iii. Read/write examples and use cases

We tested the simulated view with a simple query:

```

SELECT * FROM customer_orders_json_v WHERE customer_id = 1;

```

The result shows that customer and order data are successfully combined into one JSON document:

```

{
  "customer_id": 1,
  "first_name": "Alice",
  "last_name": "Mason",
  "email": "alice@mail.com",
  "orders_json": [
    {"order_id": 1, "order_date": "2025-11-07", "total_amount": 9.48},
    {"order_id": 2, "order_date": "2025-11-07", "total_amount": 15.48}
  ]
}

```

Through this demonstration, we proved that even without Oracle 23ai's full duality functionality, we could replicate its main concept using standard SQL functions. This approach allows us to bridge structured OLTP data and semi-structured JSON data, enabling hybrid queries and preparing the system for future Oracle 23ai upgrades.

6. Use Case Demonstrations

i. Realistic scenarios the system supports (e.g., customer analytics, clinical decision support)

The Smart Retail Analytics Hub supports both operational and analytical decision-making by combining the OLTP schema, OLAP cube, and JSON document store.

- Real-time KPI dashboard for store managers: Query 6.1 counts today's orders, revenue, and average order value, allowing daily performance tracking without delay
- Inventory status and stock alerts: Query 6.2 classifies stock levels (CRITICAL, LOW, ADEQUATE) by joining inventory, stores, and products. It prevents stockouts by flagging items under 100 units
- Store performance comparison: Query 6.3 summarizes orders and revenue by store and location for the current day, highlighting which branches generate the highest sales
- Category sales dashboard: Query 6.4 aggregates product performance by category and computes each category's revenue share percentage, giving executives a clear picture of which segments drive profit
- Customer segmentation and AI targeting: Queries 6.6 and 6.8 analyze each customer's purchase frequency and lifetime value, label segments as HIGH-, MEDIUM-, or LOW-VALUE, and link them with JSON-stored preferences to personalize offers
- Recommendation and forecast use cases: Queries 6.5 and 6.7 mine basket patterns ("frequently bought together") and generate next best product recommendations based on purchase similarities between customers
- Real-time alerts for low stock & high demand: Query 6.9 projects 30-day demand from 7-day sales and flags items that are likely to run out soon

ii. Sample queries combining relational, OLAP, and document data

- Operational KPI (OLTP): Provides instant insight into daily performance

```
SELECT COUNT(*) AS orders_today,  
       ROUND(SUM(total_amount),2) AS revenue_today,  
       ROUND(AVG(total_amount),2) AS avg_order_value  
FROM orders  
WHERE TRUNC(order_datetime)=TRUNC(SYSDATE);
```

- Category Performance (OLAP): Summarizes sales by category for trend analysis.

```
SELECT p.category,  
       SUM(oi.quantity) AS total_units,  
       ROUND(SUM(oi.quantity*oi.unit_price),2) AS category_revenue  
FROM order_items oi  
JOIN products p ON oi.product_id=p.product_id  
GROUP BY p.category;
```

- Customer Segments + JSON Preferences: Joins customer JSON profiles with product catalog to create personalized recommendations and loyalty discounts

```
SELECT JSON_VALUE(cp.profile_data,'$.name') AS customer_name,
       jt.preferred_category,
       p.product_name,
       CASE
         WHEN JSON_VALUE(cp.profile_data,'$.loyalty_points')>1000
         THEN ROUND(p.unit_price*0.9,2)
         ELSE p.unit_price
       END AS recommended_price
FROM customer_profiles cp
CROSS JOIN JSON_TABLE(cp.profile_data,'$.preferences.categories[*]'
  COLUMNS (preferred_category VARCHAR2(50) PATH '$')) jt
JOIN products p ON p.category=jt.preferred_category;
```

iii.Screenshots or outputs from Oracle 23ai worksheet

	PRODUCT_A		PRODUCT_B	TIMES_BOUGHT_TOGETHER
1	Yogurt 500g		Green Tea 250g	2
2	Apple Juice 1L		Coffee Beans 500g	1
3	Apple Juice 1L		Granola Bar	1
4	Cheddar Cheese 200		Potato Chips	1
5	Chocolate Bar		Cheddar Cheese 200	1
6	Chocolate Bar		Orange Juice 1L	1
7	Coffee Beans 500g		Milk 1L	1

To identify cross-selling opportunities, we implemented the “frequently bought together” query. This SQL joins the order_items table to itself to find pairs of products that appear in the same orders. The resulting table reveals how often two products are purchased together, measured by the times_bought_together count.

From the output, the strongest association occurs between Yogurt 500g and Green Tea 250g, which were bought together twice. Other common pairs include Apple Juice 1L + Granola Bar and Chocolate Bar + Cheddar Cheese 200g. These insights allow the retailer to create bundle deals, shelf placements, or recommendation engine inputs that promote products commonly purchased together.

	STORE_NAME	PRODUCT_NAME	CATEGORY	QTY_ON_HAND	STOCK_STATUS
1	Super Store	Cheddar Cheese 200g	dairy	70	LOW
2	ShopEZ	Yogurt 500g	dairy	80	LOW
3	BudgetMart	Orange Juice 1L	beverages	90	LOW
4	Super Store	Chocolate Bar	snacks	90	LOW
5	DailyShop	Yogurt 500g	dairy	95	LOW
6	FamilyStore	Green Tea 250g	beverages	95	LOW
7	MegaStore	Coffee Beans 500g	beverages	100	ADEQUATE
8	The Store	Apple Juice 1L	beverages	100	ADEQUATE
9	FreshMart	Orange Juice 1L	beverages	110	ADEQUATE
10	DailyShop	Green Tea 250g	beverages	120	ADEQUATE

The inventory status query works by combining data from three main tables: inventory, stores, and products. It joins these tables together to match each product's quantity on hand (qty_on_hand) with its corresponding store and category information.

After joining, the query uses a CASE statement to classify the stock level for each product into three categories. This approach allows the system to automatically flag low-stock items so managers can quickly identify which products need restocking. For example, Cheddar Cheese 200g and Yogurt 500g are labeled as "LOW" in the table because their quantities are under 100, while Coffee Beans 500g and Orange Juice 1L are marked "ADEQUATE." By running this query regularly, store managers can monitor inventory health in real time and make faster, data-driven decisions about replenishment and logistics.

	CUSTOMER_NAME	EMAIL	LOYALTY_POINTS	PREFERRED_CATEG	PRODUCT_NAME	UNIT_PRICE	RECOMMENDED_PRICE
1	Alice Mason	alice@mail.com	1700	beverages	Coffee Beans 500g	12.99	11.69
2	Alice Mason	alice@mail.com	1700	beverages	Green Tea 250g	8.49	7.64
3	Alice Mason	alice@mail.com	1700	beverages	Orange Juice 1L	3.59	3.23
4	Alice Mason	alice@mail.com	1700	snacks	Chocolate Bar	1.25	1.13
5	Alice Mason	alice@mail.com	1700	snacks	Potato Chips	2.29	2.06
6	Bob Khan	bob@mail.com	800	dairy	Cheddar Cheese 200g	4.99	4.99
7	Bob Khan	bob@mail.com	800	dairy	Yogurt 500g	3.1	3.1

The table above demonstrates how the system uses JSON-stored customer preferences to generate personalized product recommendations. Each customer's profile in the customer_profiles table contains fields such as loyalty_points and preferences, including their favorite product categories.

Using a query that combines the customer_profiles and products tables, the system reads each customer's preferred categories from the JSON data. It then lists all products from those categories that the customer hasn't purchased yet. The RECOMMENDED_PRICE column applies a small discount (10%) for customers with over 1,000 loyalty points, rewarding loyal shoppers automatically. This logic comes directly from the SQL query using a conditional case expression.

Overall, this output shows how the Smart Retail Analytics Hub leverages JSON data and SQL logic to deliver personalized offers, helping the business improve customer satisfaction and increase sales through targeted recommendations.

7. Challenges and Solutions

The development of Smart Retail Analytics Hub was exciting yet we still faced several issues. However, it challenged our technical adaptability and teamwork.

i. Technical or design challenges encountered

Throughout the whole work, all of us encountered the same persistent issue - instability of Oracle Live SQL. The platform frequently froze or disconnected during code manipulation, which resulted in lost progress. It forced us to rewrite the same code again and again, which also postponed the project completion. Constant interruptions also made it difficult to test our queries continuously.

Another challenge involved automation of the time dimension in the OLAP model. The initial plan was to manually populate the Dim_Time table with date records, but we constantly encountered errors in the system. One of the possible ways to address this issue was an attempt to extract date information directly from the order_datetime column in the Orders table but we didn't succeed as several analytical queries initially failed since we changed how the data was joined.

Lastly, version control was the most significant challenge. Since we worked on the same Oracle environment, our scripts often depended on each other, and running them out of order or overwriting changes caused conflicts and missing tables.

ii,iii. How the team resolved them and what was learnt

To address the problem with system freezes, we decided to start saving our code externally after each execution and frequently switched between regular and incognito browser modes to minimize cache-related crashes. These extra steps required more effort as we had to be constantly alerted that the system might suddenly crash. However, they ensured that our progress was saved, and it allowed us to continue developing consistently. This experience taught us the importance of patience, and surely, be prepared for potential technical disruptions when using online database tools.

To fix the time automating issue, we changed our queries to use TRUNC(order_datetime) and adjusted the joins in the fact table to match the new structure. Such an implementation improved the accuracy of our analyses and streamlined the ETL process. When dealing with this issue, we learned that automating repetitive data processes reduces the likelihood of human error and strengthens the overall scalability of the system.

Finally, we solved the collaboration challenge by creating a shared drive where we uploaded timestamped script versions and process updates. We also made sure to delegate responsibilities for each part clearly and fairly. These practices helped us maintain organization, reduce errors, and keep all team members aligned. This experience emphasized the value of clear communication and systematic version control, even in a small academic project.

8. Conclusion

i. Summary of achievements

The Smart Retail Analytics Hub successfully integrates three major data paradigms relational OLTP, analytical OLAP, and semi-structured JSON storage within a unified Oracle 23ai-compatible

ecosystem. The team built and populated normalized transactional tables, designed an optimized star schema for analytical processing, and implemented JSON document stores and simulated JSON Relational Duality Views to bridge structured and unstructured data. Together, these components form a complete data pipeline capable of supporting both real-time operational needs and long-term strategic insights. The project demonstrates that it is possible to create a hybrid retail data environment capable of driving advanced reporting, predictive analytics, and business intelligence dashboards.

ii. How the project demonstrates integration of course skills

This project serves as a practical culmination of the key skills taught in ISM 4545: database design, data warehousing, big data architecture, and analytics integration. The OLTP schema applied normalization and referential integrity principles from relational modeling. The OLAP star schema and data cubes showcased dimensional modeling, aggregation, and hierarchical querying. JSON storage and duality views reflected the modern shift toward hybrid database systems and unstructured data management. Additionally, the team's use of Oracle SQL features such as joins, aggregations, JSON_OBJECT, and JSON_ARRAYAGG demonstrated proficiency in SQL-based analytical querying, data transformation, and visualization core skills in big data analytics for AI-driven environments.

iii. Potential extensions or future enhancements

Looking forward, the Smart Retail Analytics Hub can be extended in several directions:

Integration with BI Dashboards: Connecting the Oracle backend with Power BI or Tableau for real-time visualization and decision-making.

Machine Learning Extensions: Using the integrated data layers to develop predictive models for customer churn, demand forecasting, and recommendation systems.

Automation and ETL Pipelines: Implementing automated data extraction, transformation, and loading (ETL) to refresh the OLAP and JSON layers continuously.

API Development: Expanding the JSON Duality Views into full RESTful APIs for external applications and web dashboards.

Scalability Enhancements: Migrating the system to Oracle Cloud Infrastructure or similar cloud services for larger data volumes and enterprise-level performance.

These future directions would elevate the current hybrid data system into a fully operational big data analytics platform scalable, AI-ready, and aligned with modern industry standards for smart retail management.

9. Appendix

i, ii. Full SQL code and scripts/ JSON sample documents

 SMART RETAIL ANALYTICS HUB - FINAL

ii. iii. Screenshots of dashboards or query results



10. References

i. Any external resources, documentation, literature or tutorials used to support the project (APA 7th edition style guide for all formatting)

Oracle Corporation. (2024). *Oracle Live SQL: Cloud-based SQL worksheet and tutorials*. Oracle. <https://livesql.oracle.com/landing/>

Canva Pty Ltd. (n.d.). *[Online design]*. Canva. https://www.canva.com/design/DAG4Ksgmc0g/EokHamJK0OpY4SF3D6YrSw/edit?utm_content=DAG4Ksgmc0g&utm_campaign=designshare&utm_medium=link2&utm_source=sharebutton

Sathi, A. (2012). *Big data analytics: Disruptive technologies for changing the game*. MC Press Online, LLC.